MLPy Workshop 8

March 11, 2026

1 Week 8 - Trees and Ensembles

YuTing Xie (preferred: Ella Xie)

1.0.1 Aims

By the end of this notebook you will be able to understand:

- The Basics of Decision Trees
- Cost Complexity Pruning
- Ensembles (bagging, random forests, voting)

1. Setup
2. Decision Trees
3. Ensembles

As usual, during workshops, you will complete the worksheets together in teams of 2-3, using **pair programming**. When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.
- In some Exercises, you will see some hints at the bottom of questions.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday of the week the workshop was given.

2 Setup

Let's load the packages needed for this workshop (more will be loaded through the workshop).

[1]:
```
# Display plots inline
%matplotlib inline

# Data libraries
import pandas as pd
import numpy as np
```

```python
# Plotting libraries
import matplotlib.pyplot as plt
import seaborn as sns

# sklearn modules
import sklearn
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# some new libraries and modules for this week
from sklearn.tree import export_graphviz, DecisionTreeClassifier
```

3 Introduction to Decision Trees

Decision trees provide a simple, interpretable model for both classification and regression tasks. They are able to capture nonlinear relationships between the features and target variables, by learning simple decision rules and assuming piecewise constant approximations. The deeper the tree, the more complex the decision rules and the better the fit on the training data.

Decision trees require little preprocessing and feature engineering. While other methods often require standardization, this is not the case for trees, and some tree and algorithm combinations can even support missing values. Decision trees able to handle both numerical and categorical data. However, sklearn does not implent more advanced methods for multi-class categorical features, and one-hot encoding must be used; if the number of features is large, this may result in many features and complex trees that suffer from overfitting.

To learn the decision tree, sklearn uses an optimized version of the Classification and Regression Tree (CART) algorithm, which is a greedy algorithm where locally optimal decisions are made at each node, starting from the root down to the leaves. At each node/step, CART uses binary spliting and finds the threshold yielding the largest information gain. For more information on sklearn's implementation of decision trees, see https://scikit-learn.org/stable/modules/tree.html#tree-classification.

The sklearn implementation supports both regression tasks through `DecisionTreeRegressor` and classication through `DecisionTreeClassifier`. In this workshop, we will focus on **classification**.

Let's start by loading data to explore the `DecisionTreeClassifier`.

3.1 Palmer Penguin Data

The palmer penguins dataset contains information for 344 penguins from 3 different species and from 3 islands in the Palmer Archipelago, Antarctica.

[2]:
```python
# Load the data
penguins = sns.load_dataset("penguins")
penguins.head()
```

[2]:
```
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen            39.1           18.7              181.0
1  Adelie  Torgersen            39.5           17.4              186.0
2  Adelie  Torgersen            40.3           18.0              195.0
3  Adelie  Torgersen             NaN            NaN                NaN
4  Adelie  Torgersen            36.7           19.3              193.0

   body_mass_g     sex
0       3750.0    Male
1       3800.0  Female
2       3250.0  Female
3          NaN     NaN
4       3450.0  Female
```

Let's start with some basic EDA.

[3]:
```
penguins.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    object
 1   island             344 non-null    object
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    object
dtypes: float64(4), object(3)
memory usage: 18.9+ KB
```

Notice that there are some missing values. For simplicity, we will remove them.

[4]:
```
# dropna values
penguins_rm = penguins.dropna()
penguins_rm.head()
```

[4]:
```
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen            39.1           18.7              181.0
1  Adelie  Torgersen            39.5           17.4              186.0
2  Adelie  Torgersen            40.3           18.0              195.0
4  Adelie  Torgersen            36.7           19.3              193.0
5  Adelie  Torgersen            39.3           20.6              190.0

   body_mass_g     sex
0       3750.0    Male
1       3800.0  Female
```

```
2       3250.0  Female
4       3450.0  Female
5       3650.0    Male
```

We will focus on the task of classifying species. Notice that there are four numerical features and two categorical features. For simplicity, we will focus on the numerical features only. Before any further EDA, we split our data into a training and test set.

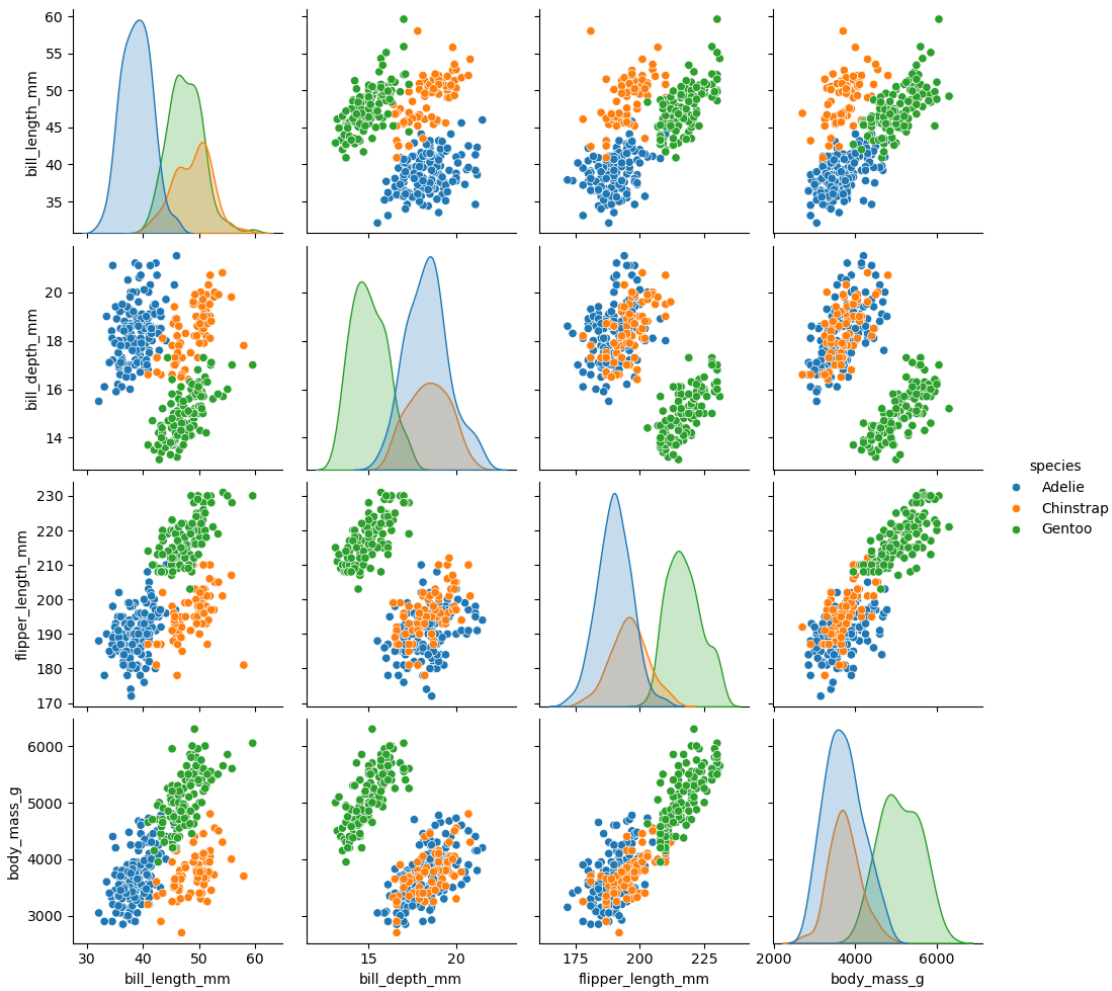
[5]:
```
# Split into train and test sets

# Feature matrix and response vector
X, y = penguins_rm.drop(["island", "sex", "species"], axis=1),␣
↪penguins_rm['species']

# Stratify split
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle= True,␣
↪test_size = 0.2, random_state=0)
```

3.1.1 Exercise 1 (CORE)

- Draw a pairplot, setting `hue = "species"`, to explore the numerical features relation with species.
- Comment on any relationships that you observe.

[6]:
```
# Code for your answer here
sns.pairplot(penguins_rm,hue='species')
```

[6]:
```
<seaborn.axisgrid.PairGrid at 0x7fbcd1af3aa0>
```



The features of Gentoo show stronger correlations among themselves. Adelie and CHinstrap have certain similarities in their characteristics.

3.2 Classification Trees

Recall from lectures, the decision tree has a number of parameters that affect the shape of the tree:

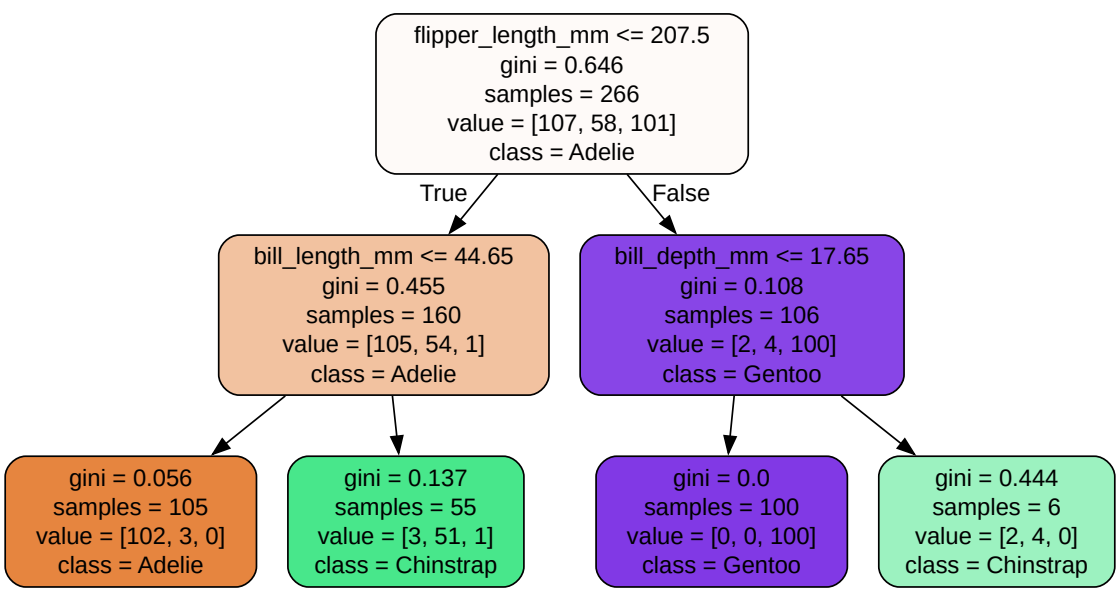
- `max_depth` : the maximum depth of the tree.
- `min_samples_split` : the minimum number of samples required to split an internal node
- `min_samples_leaf`: the minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least min_samples_leaf training samples in each of the left and right branches.
- `max_leaf_nodes`: the maximum number of leaf nodes.
- `min_impurity_decrease`: A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

As usual, once we create our `DecisionTreeClassifier` object, we can fit the tree by calling `.fit()`. Once fit, the tree will have a number of attributes and we can call additional methods such as `.predict()` or `.predict_proba()` to predict hard or soft labels, respectively, for any new data.

Let's fit our first decision tree. In the following code, we fit a simple tree of depth 2 by specifying `max_depth=2`.

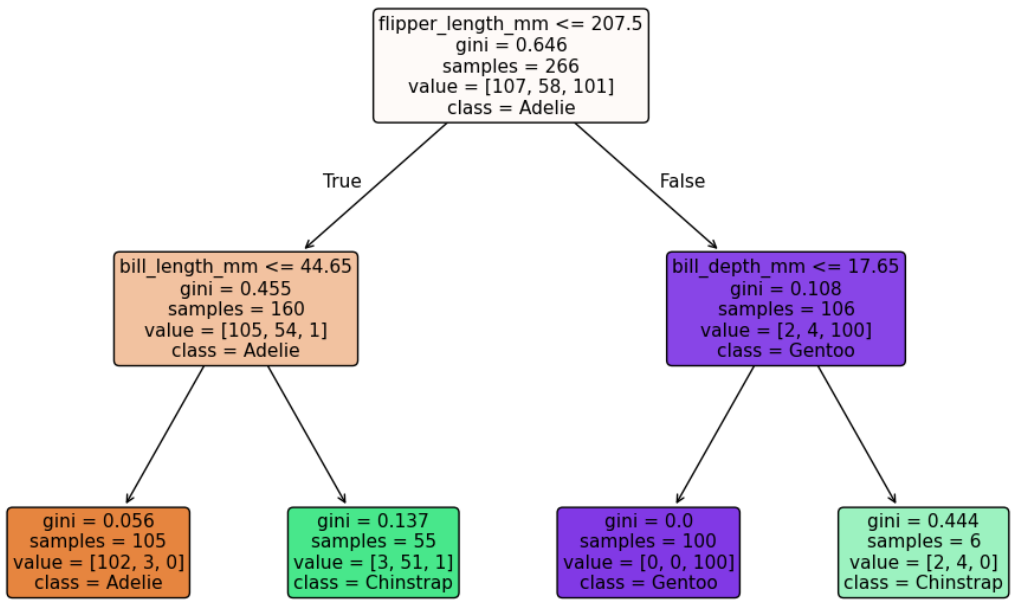
[7]:
```
# If you need to install graphviz
#! pip install graphviz
import graphviz

# Decsion tree with max depth of 2
tree_clf = DecisionTreeClassifier(max_depth = 2)
tree_clf.fit(X_train, y_train)

# Plot the decision tree
tree_data = export_graphviz(
 tree_clf,
 rounded = True, filled = True, feature_names=X_train.columns, class_names =␣
 ↪tree_clf.classes_
 )
graphviz.Source(tree_data)
```

[7]:



NOTE: If you working locally and inkscape is not installed, you will have issues in exporting the notebook to PDF when graphviz images are included. In this case, please either install inkscape or use the `plot_tree` function below to plot the tree instead.

[8]:
```
from sklearn.tree import plot_tree

plt.figure(figsize=(12,8))
plot_tree(tree_clf, rounded = True, filled = True, feature_names=X_train.
 ↪columns, class_names = tree_clf.classes_)
plt.show()
```



3.2.1 Exercise 2 (CORE)

Below, we print out the first five rows of the test data. Based on the fitted tree above (not using `.predict()`!!), what is the classified species of each penguin/row? what path along the decision tree is taken to classify the penguin?

[9]:
```
X_test.head()
```

[9]:

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
62	37.6	17.0	185.0	3600.0
60	35.7	16.9	185.0	3150.0
283	54.3	15.7	231.0	5650.0
107	38.2	20.0	190.0	3900.0
65	41.6	18.0	192.0	3950.0

1.flipper_length<207.5—bill_length<44.65—Adelie 2.flipper_length<207.5—bill_length<44.65—Adelie 3.flipper_length>207.5—bill_depth<17.65—Gentoo

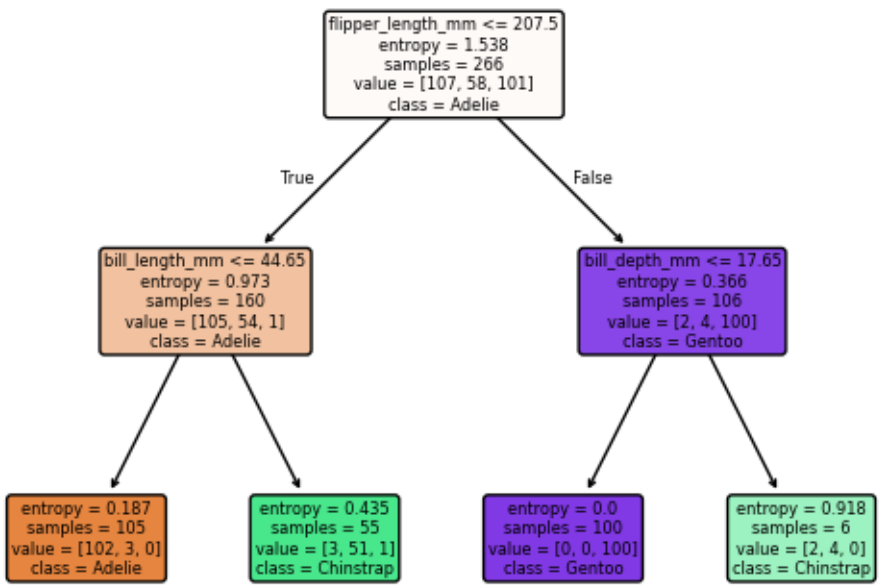
4.flipper_length<207.5—bill_length<44.65—Adelie 5.flipper_length<207.5—bill_length<44.65—Adelie

3.2.2 Exercise 3 (CORE)

Fit a new decision tree, again with a depth 2, but change the criterion to measure the quality of a split to entropy. How has the tree stucture changed?

Hint: when creating the decision tree object, set the option `criterion`.

[10]:
```
# Code for your answer here
tree_ent=DecisionTreeClassifier(max_depth=2,criterion='entropy')
tree_ent.fit(X_train,y_train)
plot_tree(tree_ent,rounded = True, filled = True, feature_names=X_train.
 ↪columns, class_names = tree_ent.classes_)
plt.show()
```



the tree structure not change.

3.2.3 Exercise 4 (CORE)

- Create two functions, one to compute the Gini index and another to compute the entropy, based on a vector of counts representing the number of observations in each category.
- For each leaf node in the trees fitted above, use your functions to compute the Gini index and entropy based on the counts reported. If your functions are correct, it should match the

numbers reported in the tree.

RECALL

- **Gini Index**: is a measure of total variance across the C classes, defined as:

$G(R_m) = \sum_{c=1}^{C} \hat{p}_{m,c}(1 - \hat{p}_{m,c}) = 1 - \sum_{c=1}^{C} \hat{p}_{m,c}^2$

where $\hat{p}_{m,c}$ is the proportion of training observations in the m th region (belonging to m th node) coming from the c th class.

- **Entropy**: is an information theoretic measure defined as:

$H(R_m) = -\sum_{c=1}^{C} \hat{p}_{m,c} \log_2 \hat{p}_{m,c}$.

[11]:
```
# Code for your answer here
def gini_index(counts):
    total=counts.sum()
    gini=1-np.sum((counts/total)**2)
    return gini
def entropy_index(counts):
    total=counts.sum()
    pro=counts/total
    pro=pro[pro>0]
    entropy=-np.sum(pro*np.log2(pro))
    return entropy

leaf_nodes = np.where((tree_clf.tree_.children_left == -1) &
                      (tree_clf.tree_.children_right == -1))[0]
for i in leaf_nodes:
    counts=tree_clf.tree_.value[i][0]
    print(gini_index(counts))

leaf_nodes1 = np.where((tree_ent.tree_.children_left == -1) &
                      (tree_ent.tree_.children_right == -1))[0]
for i in leaf_nodes1:
    counts=tree_ent.tree_.value[i][0]
    print(entropy_index(counts))
```

```
0.05551020408163265
0.13685950413223147
0.0
0.4444444444444444
0.18717625687320816
0.4350218963844825
-0.0
0.9182958340544896
```

Note that our fitted `DecisionTreeClassifier` stores the tree as an attribute, which can be accessed by calling `.tree_`. For more information on how `sklearn` stores a tree, see here. The fitted tree has a number of attributes, including `.impurity` providing the impurity for all nodes.

[12]:
```
# The impurity values for all nodes are accessible in the impurity attribute of␣
↪the tree.
tree_clf2=tree_ent
print(tree_clf.tree_.impurity)
print(tree_clf2.tree_.impurity)
```

```
[0.64647521 0.45539063 0.0555102  0.1368595  0.10822357 0.
 0.44444444]
[1.5380673  0.97342994 0.18717626 0.4350219  0.36579197 0.
 0.91829583]
```

3.2.4 Exercise 5 (CORE)

Trying increasing the maximum depth to 10.

- Fit and visualize the tree.
- What is the depth of the tree? **Hint**: check the available methods for a `DecisionTreeClassifier`.
- How does the accuracy on both the train and test data compare with our previous tree with a depth of 2?

[13]:
```
# Code for your answer here
from sklearn.metrics import accuracy_score
tree_clf3=DecisionTreeClassifier(max_depth=10,criterion='entropy')
tree_clf3.fit(X_train,y_train)
plot_tree(tree_clf3,rounded = True, filled = True, feature_names=X_train.
↪columns, class_names = tree_clf3.classes_)
plt.show()
print(tree_clf3.get_depth())
test1_ac = accuracy_score(y_test, tree_clf.predict(X_test))
test3_ac = accuracy_score(y_test, tree_clf3.predict(X_test))
print(test1_ac)
print(test3_ac)
```

```
6
0.9402985074626866
0.9253731343283582
```

the accuracy is better than depth 2.

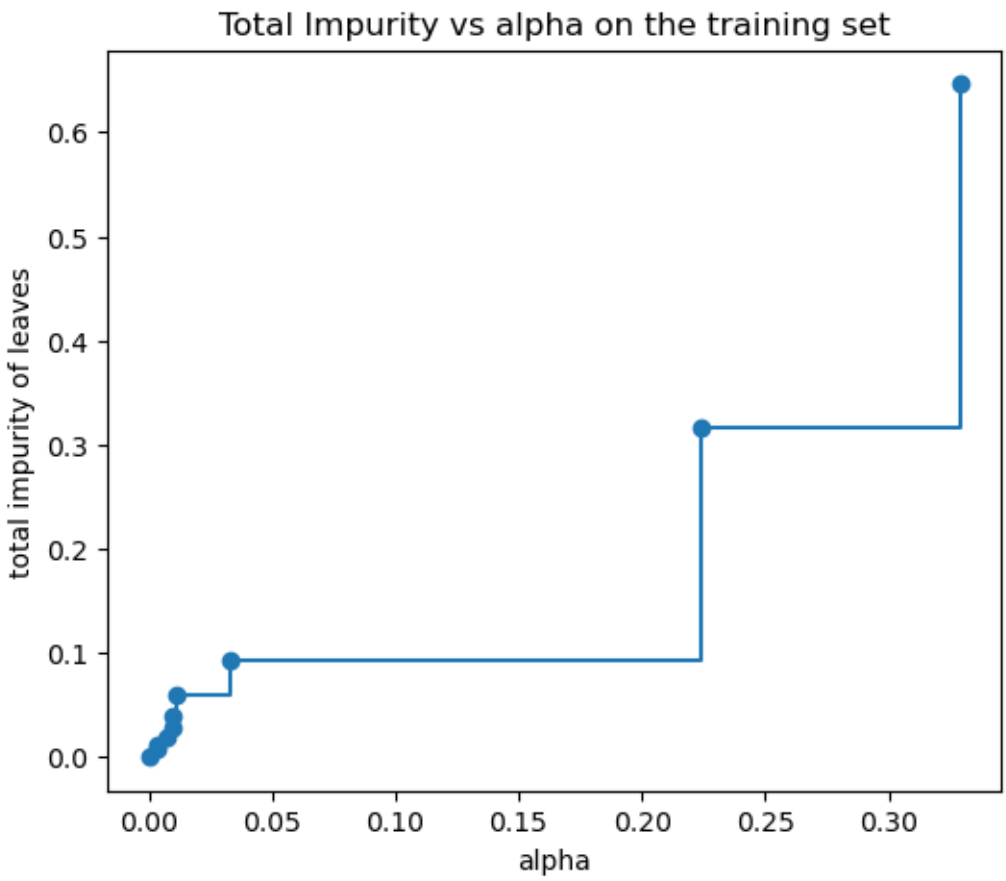
3.3 Cost Complexity Pruning

Instead of using cross-validation to fit and tune the tree depth and shape across a range of values, a more effective and computationally efficient strategy is **cost complexity pruning**. Recall that in cost complexity pruning, we start by constructing a complex tree, and sequentially prune back the tree by snipping off the branches that minimize the cost complexity. This produces a sequence of nested trees, with each one associated to range of values of α (i.e. each tree is the optimal, unique tree for this range of α).

We can perform cost complexity pruning in `sklearn` by using the method `cost_complexity_pruning_path()` on our decision tree object. Recall that a larger value of α implies more regularization and more shallow trees.

[14]:
```
# Cost complexity pruning
tree_clf = DecisionTreeClassifier()
tree_clf.fit(X_train, y_train)
path = tree_clf.cost_complexity_pruning_path(X_train,y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
```

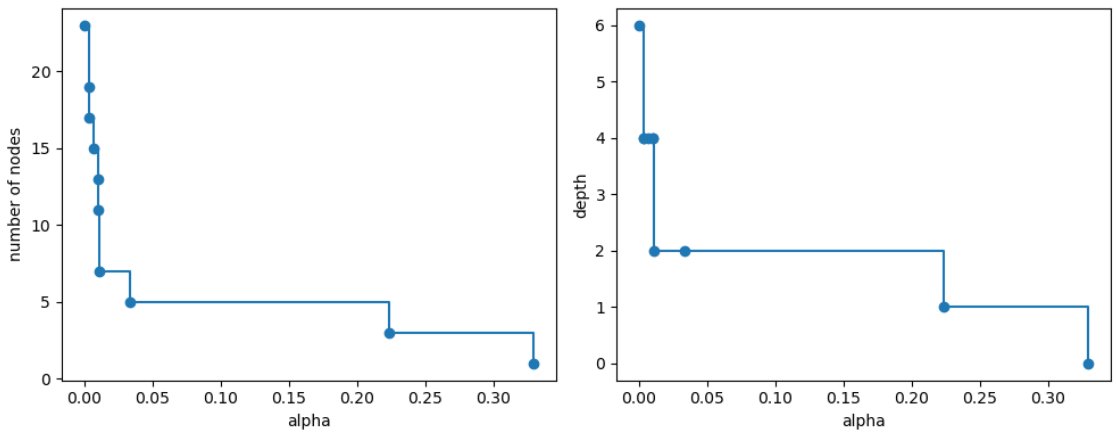
```
fig, ax = plt.subplots(figsize=(6,5))
ax.plot(ccp_alphas, impurities, marker="o", drawstyle="steps-post")
ax.set_xlabel("alpha")
ax.set_ylabel("total impurity of leaves")
ax.set_title("Total Impurity vs alpha on the training set")
plt.show()
```



Run the following code to understand the depth and number of nodes in each tree in the sequence.

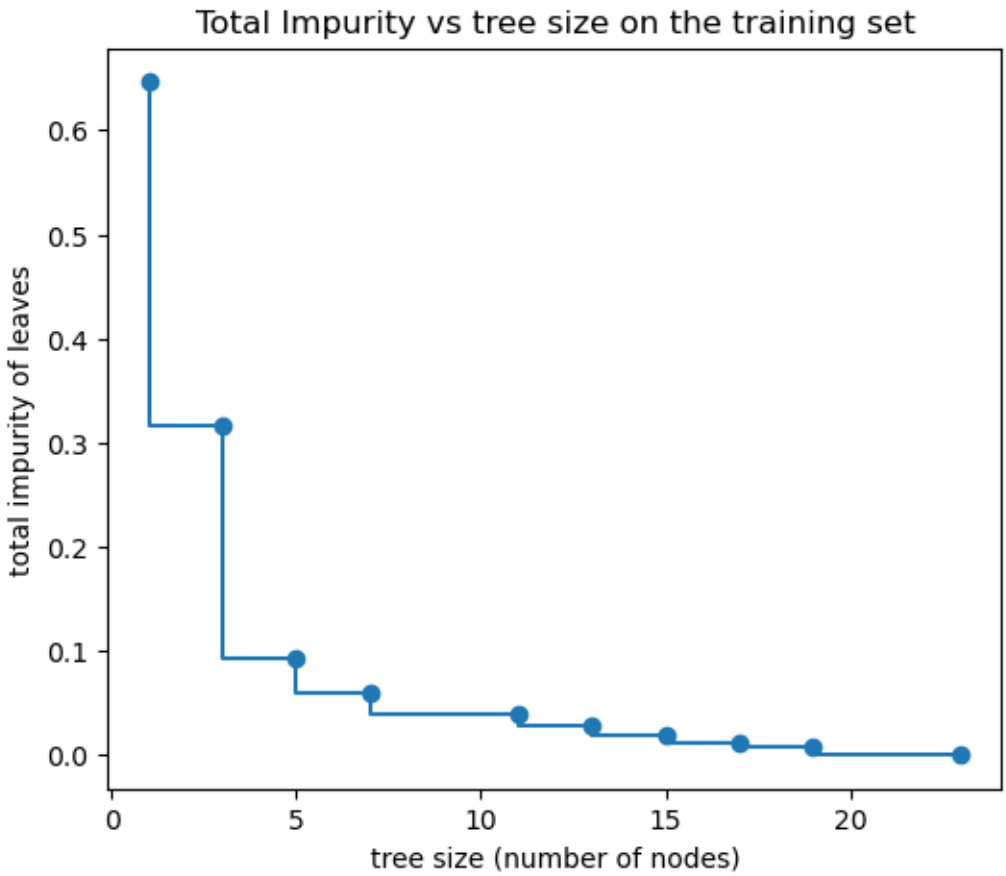
[15]:
```
# Fit and store the decision tree for each value of alpha
clfs = []
for ccp_alpha in ccp_alphas:
    clf = DecisionTreeClassifier(ccp_alpha=ccp_alpha)
    clf.fit(X_train, y_train)
    clfs.append(clf)

# Plot the depth and number of nodes for each tree for in the sequence as a␣
↪function of alpha
node_counts = [clf.tree_.node_count for clf in clfs]
```

```
depth = [clf.tree_.max_depth for clf in clfs]
fig, ax = plt.subplots(1,2, figsize=(10,4))
ax[0].plot(ccp_alphas, node_counts, marker="o", drawstyle="steps-post")
ax[0].set_xlabel("alpha")
ax[0].set_ylabel("number of nodes")
ax[1].plot(ccp_alphas, depth, marker="o", drawstyle="steps-post")
ax[1].set_xlabel("alpha")
ax[1].set_ylabel("depth")
fig.tight_layout()
```



We see that each α is associated with unique number of nodes and increasing α reduces the size of tree (number of nodes). Thus, it is often more intutive to plot the total impurity as a function of the tree size.

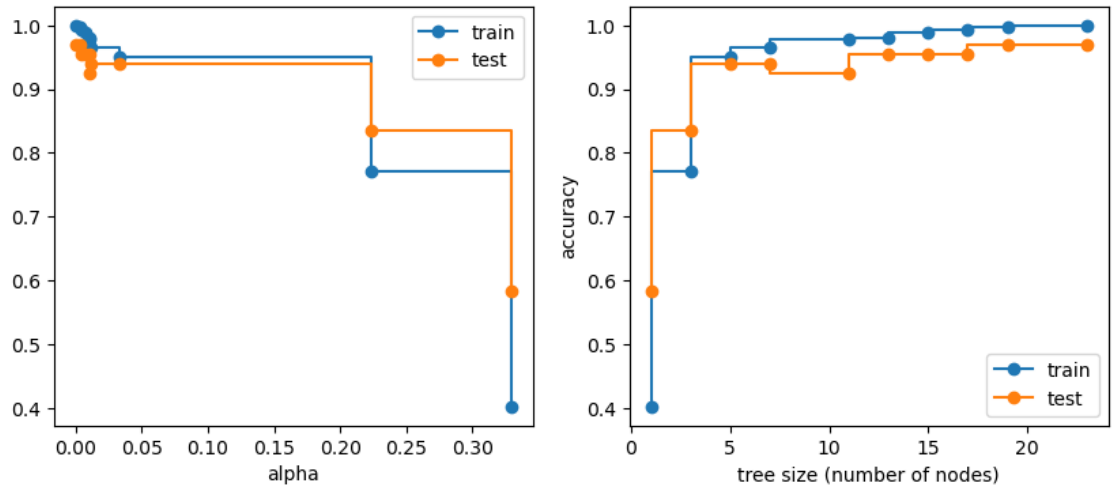
[16]:
```
fig, ax = plt.subplots(figsize=(6,5))
ax.plot(node_counts, impurities, marker="o", drawstyle="steps-post")
ax.set_xlabel("tree size (number of nodes)")
ax.set_ylabel("total impurity of leaves")
ax.set_title("Total Impurity vs tree size on the training set")
plt.show()
```



Run the following code to compute the accuracy of each tree in the sequence on both the training and test set.

[17]:
```
train_scores = [clf.score(X_train, y_train) for clf in clfs]
test_scores = [clf.score(X_test, y_test) for clf in clfs]

fig, ax = plt.subplots(1,2,figsize=(10,4))
ax[0].set_xlabel("alpha")
ax[0].plot(ccp_alphas, train_scores, marker="o", label="train",␣
 ↪drawstyle="steps-post")
ax[0].plot(ccp_alphas, test_scores, marker="o", label="test",␣
 ↪drawstyle="steps-post")
ax[0].legend()
ax[1].set_xlabel("tree size (number of nodes)")
ax[1].set_ylabel("accuracy")
ax[1].plot(node_counts, train_scores, marker="o", label="train",␣
 ↪drawstyle="steps-post")
ax[1].plot(node_counts, test_scores, marker="o", label="test",␣
 ↪drawstyle="steps-post")
```

```
ax[1].legend()
plt.show()
```



The training accuracy is of course a decreasing function of α (or increasing function of tree size), since the tree becomes less complex as the α increases. Instead, the test accuracy is fairly flat for any $\alpha < 0.2$ or tree size greater than 5. Of course, we can't use the test data to choose α, as that would lead to **data leakage**!! Instead, we need to use cross-validation to tune the value of α.

3.3.1 Exercise 6 (CORE)

Run the following code to compute the cross-validated accuracy across the values of α.

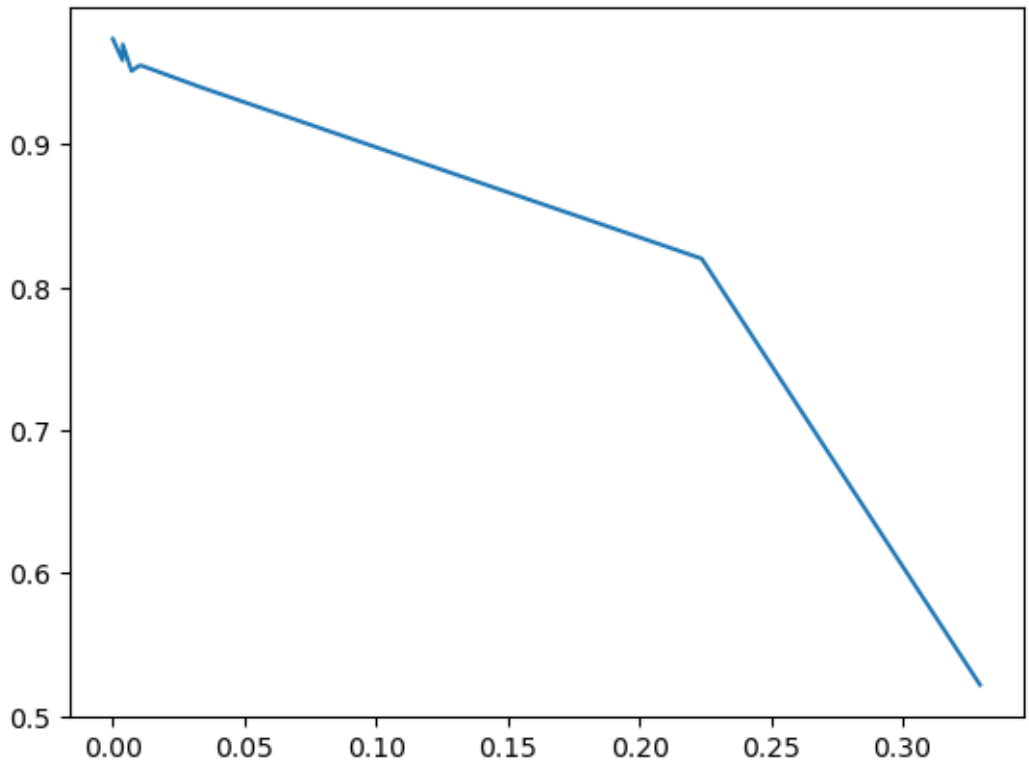
- Add the cross-validated accuracy to the figure above with train and test accuracy.
- What is best value of α returned by the grid search and the corresponding size of the tree? Based on the plot, would you choose this value or a different one? Why?

[18]:
```
# Grid search
clf = DecisionTreeClassifier()
gs = GridSearchCV(clf,
                  param_grid = {'ccp_alpha': ccp_alphas},
                  cv = KFold(5, shuffle=True, random_state=42),)
gs.fit(X_train,y_train)
```

[18]:
```
GridSearchCV(cv=KFold(n_splits=5, random_state=42, shuffle=True),
             estimator=DecisionTreeClassifier(),
             param_grid={'ccp_alpha': array([0.        , 0.0036871 , 0.0037594 ,
0.00709593, 0.01002506,
       0.01006848, 0.01095596, 0.03310162, 0.22370923, 0.32942935])})
```

[26]:
```
# Code for your answer here
cvaccuracy=gs.cv_results_["mean_test_score"]
plt.plot(ccp_alphas,cvaccuracy)
```

[26]: [<matplotlib.lines.Line2D at 0x7fbcc6f52d20>]

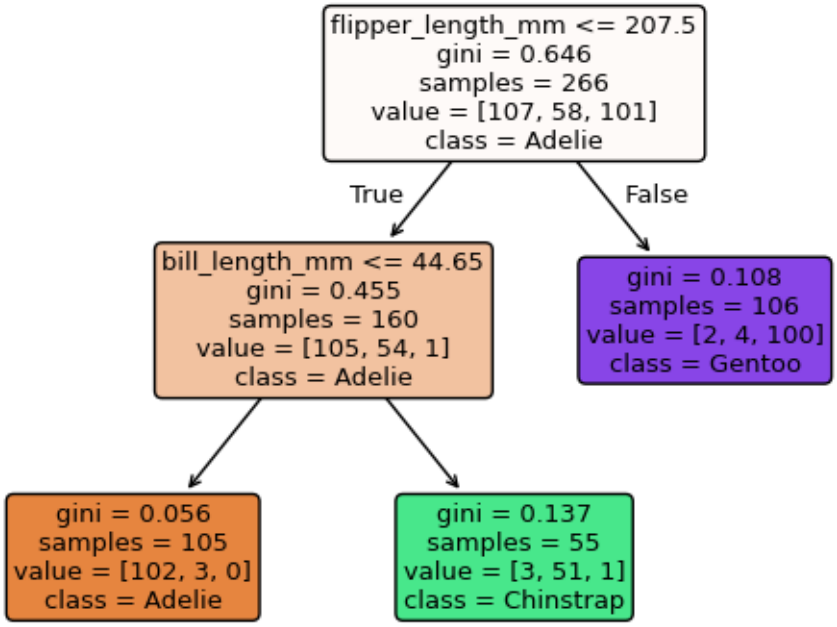


alpha=0.20, because when alpha above 0.2, the accuracy score decrease significantly.

3.3.2 Exercise 7 (CORE)

For your chosen value of α fit and visualize the tree. How does it compare to the shallow and deep tree that we initialy fit?

[27]:
```
# NOTE: call your tree clf or change to your name of the pruned tree when␣
↪computing feature importances before exercise 12

# Code for your answer
tree_pru=DecisionTreeClassifier(ccp_alpha=0.2,random_state=0)
tree_pru.fit(X_train,y_train)
plot_tree(tree_pru,rounded = True, filled = True, feature_names=X_train.
↪columns, class_names = tree_pru.classes_)
plt.show()
```



it have less nodes than other trees.

4 Ensembles

Ensembles combine the predictions of several base estimators to improve generalization and robustness, compared with a single estimator. A number of ensemble methods are available in `sklearn`, and for an overview, see https://scikit-learn.org/stable/modules/ensemble.html#. However, we only have time to cover a subset of ensemble methods, namely, **bagging**, **random forests**, and **voting**.

4.1 Bagging

A bagging classifier is an ensemble of base classifiers, each fit on random subsets of a dataset. Their predictions are then pooled or aggregated to form a final prediction. To apply bagging to decision trees, we simply construct (i) decision trees using bootstrapped training sets, (ii) agregated the predictions (either by averaging the probabilities or by majority vote).

Bagging is typically used as a way to reduce the variance of a high-variance estimator (e.g. a decision tree). The parameter `n_estimators` contols the number of bootstrapped datasets, and hence the number of estimators in the ensemble. While bagging is when sampling is performed with replacement, the `sklearn` implementation allows for more general constructions of the random subsets of the data:

- Pasting is when the sampling is done without replacement and is designed to use smaller sample sizes than the training dataset (useful when the training dataset does not fit into memory).

Pasting can be performed with `BaggingClassifier` by setting the options `bootstrap=FALSE` and `max_samples` either to an integer or a float representing the sample size of the random subset as a proportion of training points.

- Random Subspaces is when random subsets of the dataset are drawn as random subsets of the features. Random Subspaces can be performed with `BaggingClassifier` by setting the options `bootstrap=FALSE` and `max_features` either to an integer or a float representing the number of features in the random subset as a proportion of the number of features in the training data.
- Random Patches is when random subsets of the dataset are drawn based on random subsets of both the features and samples, and can be constructed by combining the options above.

Averaging methods generally work best when the classifiers are diverse, and they have been shown to give impressive improvements in accuracy by combining together hundreds or even thousands of trees into a single procedure.

Note that while the default estimator in `BaggingClassifier` is a decision tree, other classifiers can be combined by specifying the option `estimator`. Also note that in `sklearn`'s implementation, the final predictions are obtained by averaging the prediction probabilities across estimators, with majority voting only used for estimators that don't provide probabilistic outputs.

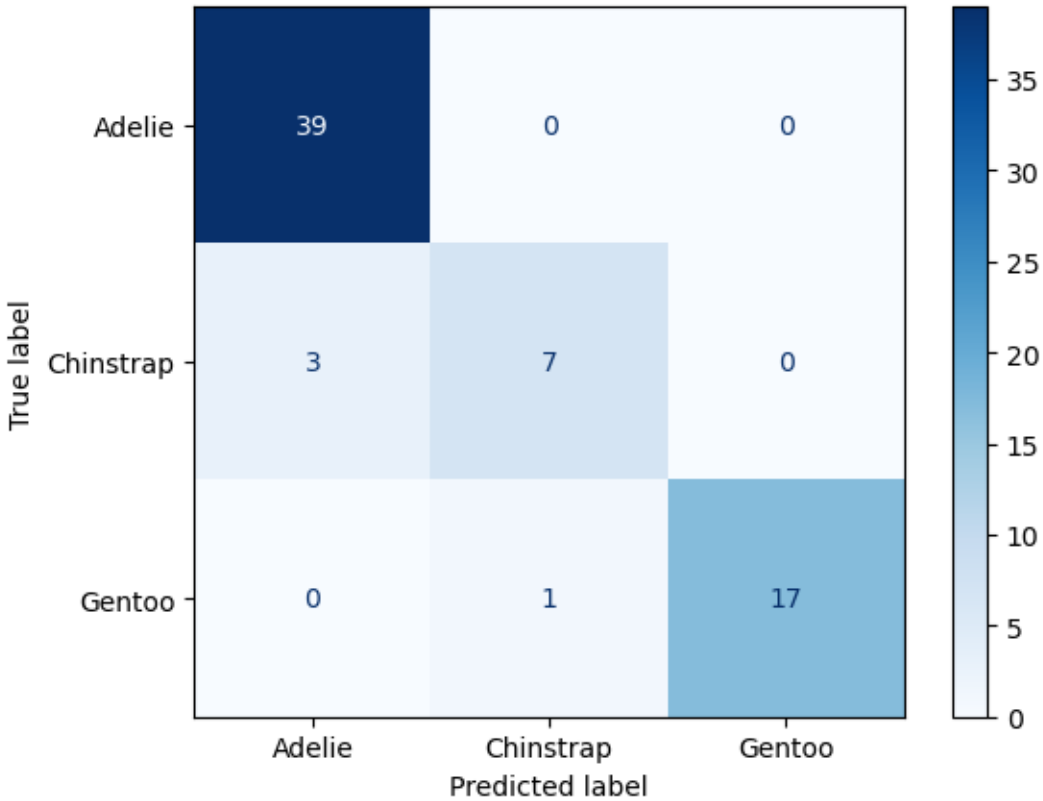
For more details, see:
https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html

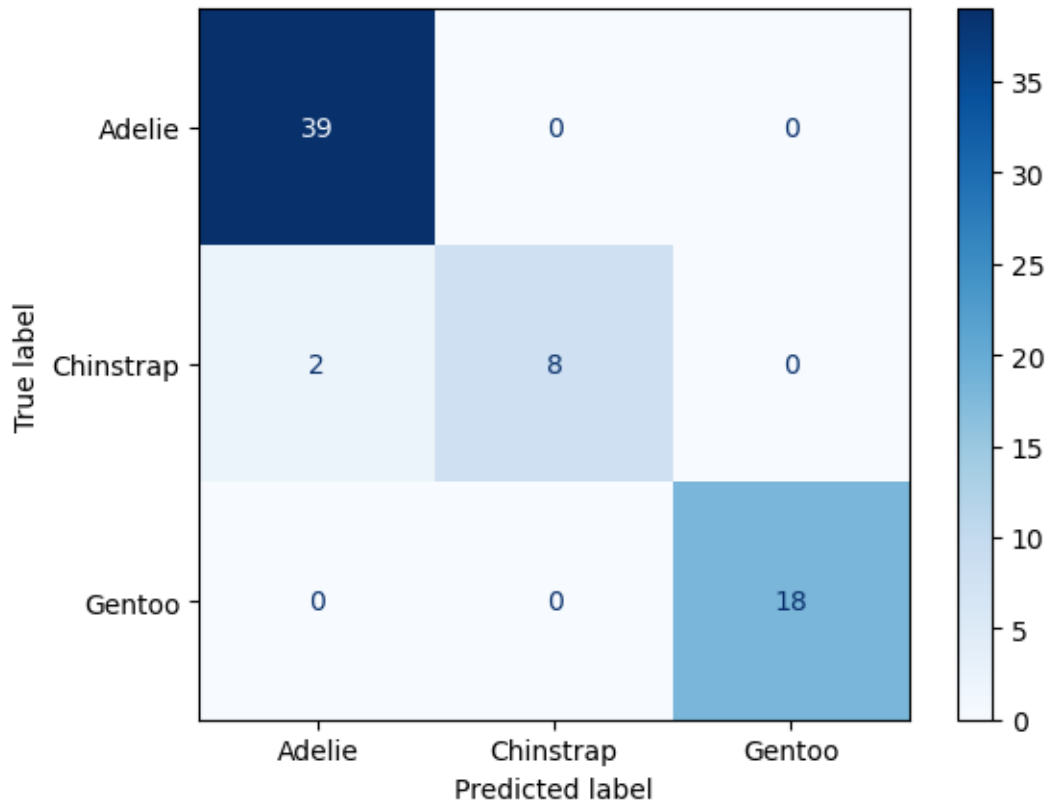
4.1.1 Exercise 8 (CORE)

- Fit a bagged classifier to the data that combines 10 decision trees.
- Plot the confusion matrix on the both test and training data for both your pruned tree and bagged classifier. Comment on the advantages and disadvantages of the pruned tree vs bagged classifier briefly.

[31]:
```
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import ConfusionMatrixDisplay
bag_tree=DecisionTreeClassifier(random_state=0)
bag=BaggingClassifier( estimator=bag_tree,
    n_estimators=10,
    bootstrap=True,
    random_state=0)
bag.fit(X_train,y_train)
ConfusionMatrixDisplay.from_estimator(tree_pru,
    X_test,
    y_test,
    cmap=plt.cm.Blues,
    display_labels=tree_pru.classes_,)
plt.show()
ConfusionMatrixDisplay.from_estimator(bag,
    X_test,
    y_test,
    cmap=plt.cm.Blues,
```

```
    display_labels=tree_pru.classes_,)
plt.show()

# Code for your answer here
```





bagged classifier is better.

4.1.2 Exercise 9 (CORE)

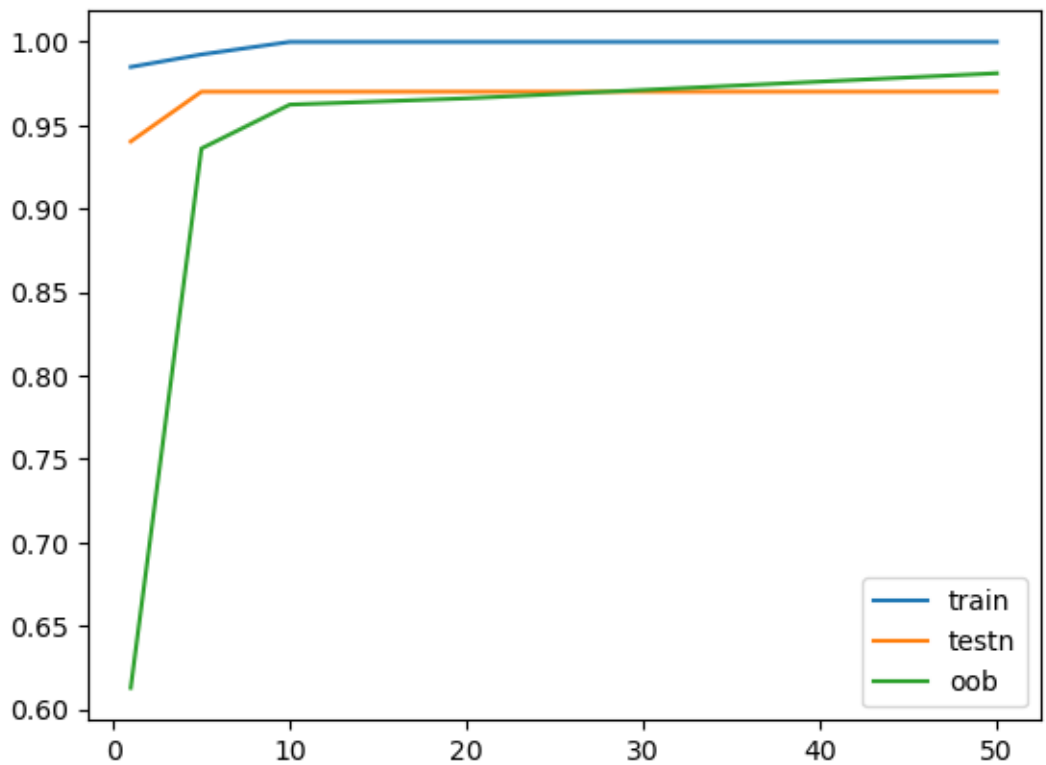
For the range of `n_estimators` provided below: - Fit the bagged trees and compute the training accuracy, test accuracy, and out-of-bag scores. - Plot all three metrics as a function of the number of estimators. - Comment on the results and suggest a suitable number of estimators for this data.

[39]:
```
n_est = np.array([1,5,10,20,50],dtype=int)
train_acc=[]
test_acc=[]
oob=[]
# Code for your answer here
for i in n_est:
    bag=BaggingClassifier( estimator=bag_tree,
    n_estimators=i,
    bootstrap=True,
    oob_score=True,
    random_state=0)
    bag.fit(X_train,y_train)
    train_acc.append(bag.score(X_train,y_train))
    test_acc.append(bag.score(X_test,y_test))
```

```
    oob.append(bag.oob_score_)
plt.plot(n_est,train_acc,label='train')
plt.plot(n_est,test_acc,label='testn')
plt.plot(n_est,oob,label='oob')
plt.legend()
plt.show()
```

```
/opt/conda/lib/python3.12/site-packages/sklearn/ensemble/_bagging.py:975:
UserWarning: Some inputs do not have OOB scores. This probably means too few
estimators were used to compute any reliable oob estimates.
  warn(
/opt/conda/lib/python3.12/site-packages/sklearn/ensemble/_bagging.py:981:
RuntimeWarning: invalid value encountered in divide
  oob_decision_function = predictions / predictions.sum(axis=1)[:, np.newaxis]
/opt/conda/lib/python3.12/site-packages/sklearn/ensemble/_bagging.py:975:
UserWarning: Some inputs do not have OOB scores. This probably means too few
estimators were used to compute any reliable oob estimates.
  warn(
/opt/conda/lib/python3.12/site-packages/sklearn/ensemble/_bagging.py:981:
RuntimeWarning: invalid value encountered in divide
  oob_decision_function = predictions / predictions.sum(axis=1)[:, np.newaxis]
```

```
/opt/conda/lib/python3.12/site-packages/sklearn/ensemble/_bagging.py:975:
UserWarning: Some inputs do not have OOB scores. This probably means too few
estimators were used to compute any reliable oob estimates.
  warn(
/opt/conda/lib/python3.12/site-packages/sklearn/ensemble/_bagging.py:981:
RuntimeWarning: invalid value encountered in divide
  oob_decision_function = predictions / predictions.sum(axis=1)[:, np.newaxis]
```



chose n_estimator=10.

4.1.3 Bagging: low diversity in estimators

Bagging alone may not produce very diverse trees, and thus may result in only modest improvements in accuracy. In particular, if there is a strong set of predictors in the data, then the bagged trees will look quite similar to each other and predictions will be highly correlated. Averaging highly correlated quantities does not lead to a large reduction in variance and provides on very small improvements in predictions.

Run the following code below, the print out the feature and threshold used for the first split for each of the trees in our ensemble. Notice how most of trees use the same first splitting rule.

[41]:
```
# print out the first split for all trees in the bagged model
for i in range(10):
    print('Estimator: ', i+1,
          ', First split feature: ', bag.estimators_[i].tree_.feature[0],
          ', First split threshold: ', bag.estimators_[i].tree_.threshold[0])
```

```
Estimator:  1 , First split feature:  2 , First split threshold:  207.5
Estimator:  2 , First split feature:  2 , First split threshold:  206.5
Estimator:  3 , First split feature:  2 , First split threshold:  206.5
Estimator:  4 , First split feature:  2 , First split threshold:  206.5
Estimator:  5 , First split feature:  0 , First split threshold:  43.25
```

```
Estimator:  6 , First split feature:  1 , First split threshold:
16.449999809265137
Estimator:  7 , First split feature:  2 , First split threshold:  207.5
Estimator:  8 , First split feature:  0 , First split threshold:
42.30000114440918
Estimator:  9 , First split feature:  2 , First split threshold:  207.0
Estimator:  10 , First split feature:  2 , First split threshold:  207.0
```

4.2 Random Forests

Random forests is one of the most widely used machine learning algorithms. A random forest is an ensemble that fits a number of decision tree classifiers on various random subsets of the data (Random Patches) and uses averaging to improve the predictive accuracy and control overfitting. Random forests are essentally bagged tree classifiers, but improve the diversity and decorrelate the trees compared with bagging by using a random sample of features each time a split in a tree is considered. The random forest algorithm can therefore be summarized as:

- Draw a random bootstrap sample of the training data.
- Grow a decision tree from the bootstrap data. At each node:
  - Randomly select a subset of features without replacement (defaults to the square root of the total number of features).
  - Split the node using the feature and threshold that provides the best split according to the objective function.
- Repeat the steps above for the desired number of bootstrapped datasets (estimators in the ensemble).
- Aggregate the predictions (`sklearn`'s `RandomForestClassifier` aggregates the prediction probabilities).

4.2.1 Parameters

Important parameters include:

- `max_features`: specifies that number of random features to consider at each split. If set to the total number of features, then it is simply bagging.
- `n_estimators`: number of trees in the ensemble; larger is generally better as averaging more trees will yield a more robust ensemble.
- `max_samples`: sample size of the bootstrap dataset (useful when the training set is large to reduce memory and computational cost).

By default, large and complex trees are grown for each bootstrapped data. While these trees tend to have high variance and overfit to the training data, aggregation reduces variance and improves predictive accuracy. If desired, the complexity of the trees can also be limited by setting the parameters of the decision trees (e.g. `max_depth`, `max_leaf_nodes`); again this may be useful to reduce memory and computational cost.

4.2.2 Exercise 10 (CORE)

- Fit a random forest classifer that includes 10 trees in the ensemble.

- Print the splitting rule (feature and threshold) for each estimator. How does this compare to the bagged model?

[43]:
```
from sklearn.ensemble import RandomForestClassifier
# Code for your answer here
rf=RandomForestClassifier(n_estimators=10,
    max_features='sqrt',
    random_state=0
)

rf.fit(X_train, y_train)
for i in range(10):
    print('Estimator: ', i+1,
          ', First split feature: ',rf.estimators_[i].tree_.feature[0],
          ', First split threshold: ', rf.estimators_[i].tree_.threshold[0])
```

```
Estimator:  1 , First split feature:  0 , First split threshold:  43.25
Estimator:  2 , First split feature:  0 , First split threshold:
43.19999885559082
Estimator:  3 , First split feature:  0 , First split threshold:
43.10000038146973
Estimator:  4 , First split feature:  1 , First split threshold:
16.399999618530273
Estimator:  5 , First split feature:  0 , First split threshold:  43.25
Estimator:  6 , First split feature:  1 , First split threshold:
16.449999809265137
Estimator:  7 , First split feature:  1 , First split threshold:
16.449999809265137
Estimator:  8 , First split feature:  0 , First split threshold:
42.30000114440918
Estimator:  9 , First split feature:  1 , First split threshold:
16.449999809265137
Estimator:  10 , First split feature:  2 , First split threshold:  207.0
```

the first node is more diverse in rf, so it can split feature more clear.

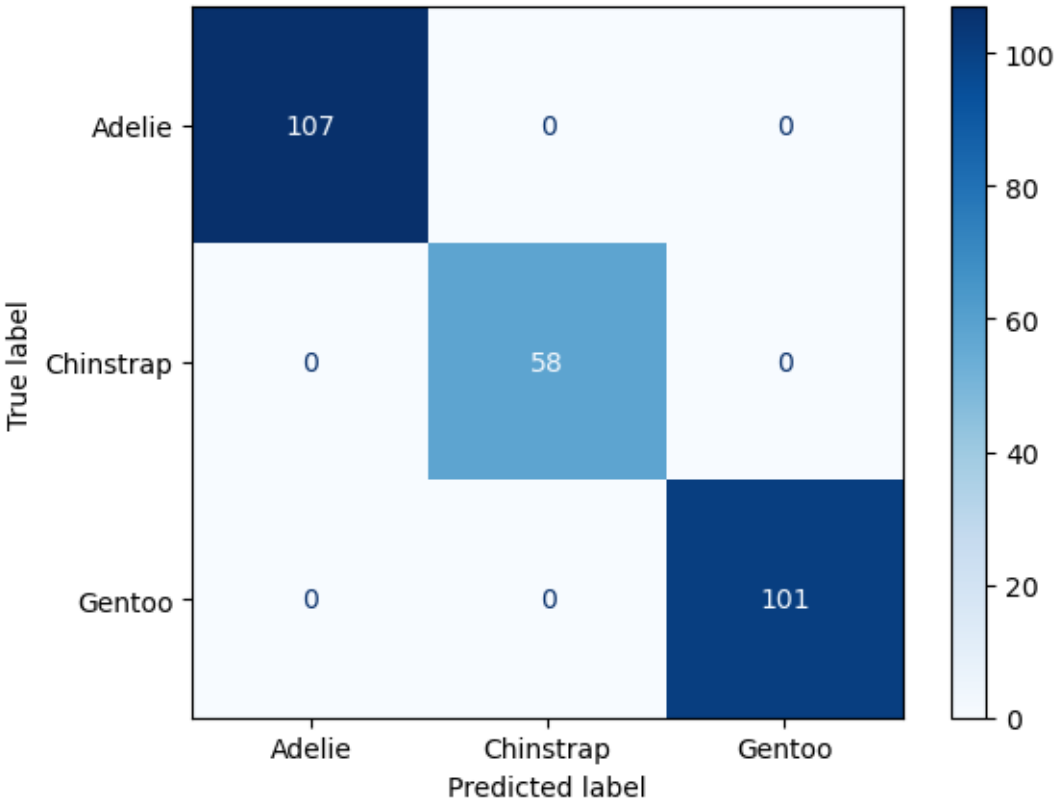
4.2.3 Exercise 11 (CORE)

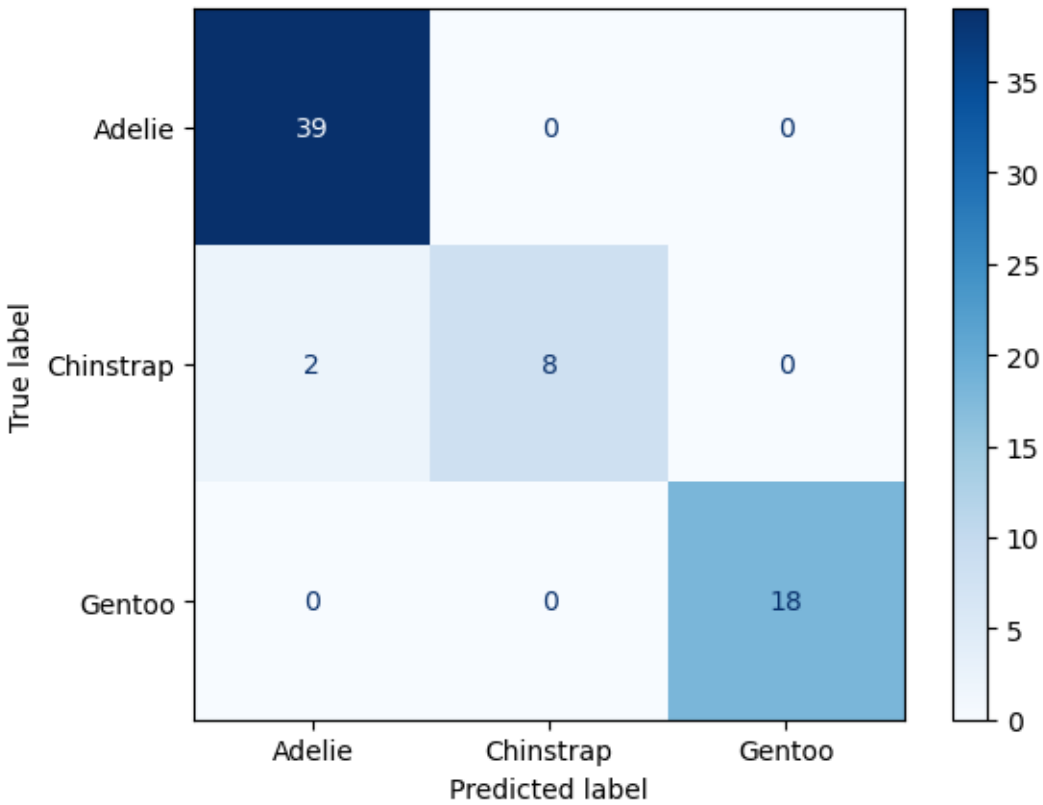
- Fit a random forest to your data, using the OOB error to tune the number of features considered at each split, for **max_features** in $[2, 3, 4]$. What is the optimal number of features?
- Visualize the confusion matrix on the train and test data.

[54]:
```
# Note: let rf be the name your random forest object with the optimal␣
↪max_features or change the name in exercise 12

# Code for your answer here
max_feature_list=[2,3,4]
oob=[]
```

```python
for i in max_feature_list:
    rf1=RandomForestClassifier(n_estimators=100,
        max_features=i,
        oob_score=True,
        random_state=0
    )
    rf1.fit(X_train, y_train)

    oob.append(1 - rf1.oob_score_)

idx = oob.index(min(oob))
max_features = max_feature_list[idx]

print(oob)
print(max_features)
ConfusionMatrixDisplay.from_estimator(rf1,
    X_train,
    y_train,
    cmap=plt.cm.Blues,
    display_labels=rf1.classes_,)
plt.show()
ConfusionMatrixDisplay.from_estimator(rf1,
    X_test,
    y_test,
    cmap=plt.cm.Blues,
    display_labels=rf1.classes_,)
plt.show()
```

```
[0.022556390977443663, 0.015037593984962405, 0.022556390977443663]
3
```


Adelie
Chinstrap
Gentoo
True label
107
0
0
0
58
0
0
0
101
Adelie
Chinstrap
Gentoo
Predicted label
100
80
60
40
20
0



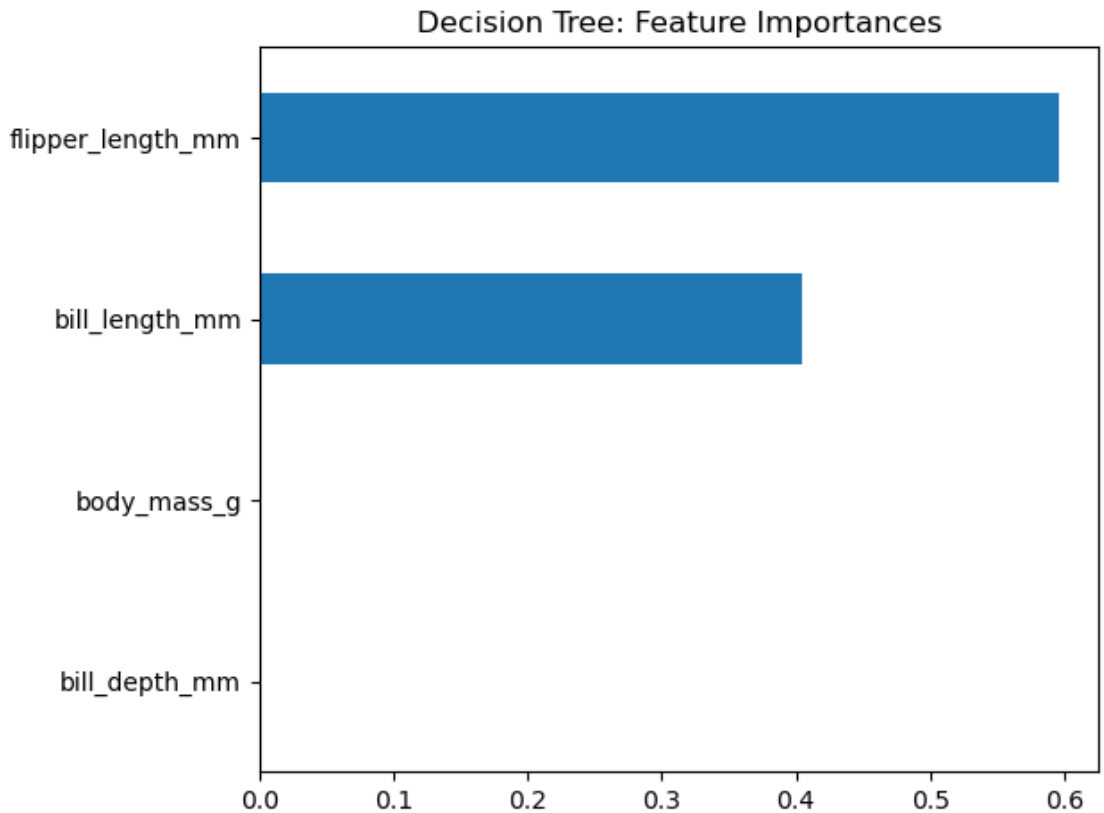
4.2.4 Feature Importance

The fitted `RandomForestClassifier` also has an attribute `feature_importances_`, that reflects the relative rank (i.e. depth) and predictive power of a feature, averaged across all trees in the ensemble. For further details, see https://scikit-learn.org/stable/modules/ensemble.html#feature-importance-evaluation, and for an example, see https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html.

Let's start by visualizing the feature importances for our pruned tree in exercise 7 (note: replace `clf` with the name of your tree from exercise 7, if not already called `clf`). Notice how this reflects the depth of the feature and impurity decrease.

[57]:
```
# NOTE: change clf to name of your pruned tree in exercise 7

# Extract feature names
feature_names = tree_pru.feature_names_in_

# Create a pandas series with the feature importances
importances = pd.Series(
    tree_pru.feature_importances_, index=feature_names
).sort_values(ascending=True)
```

```
# Plot the feature importances
ax = importances.plot.barh()
ax.set_title("Decision Tree: Feature Importances")
ax.figure.tight_layout()
```

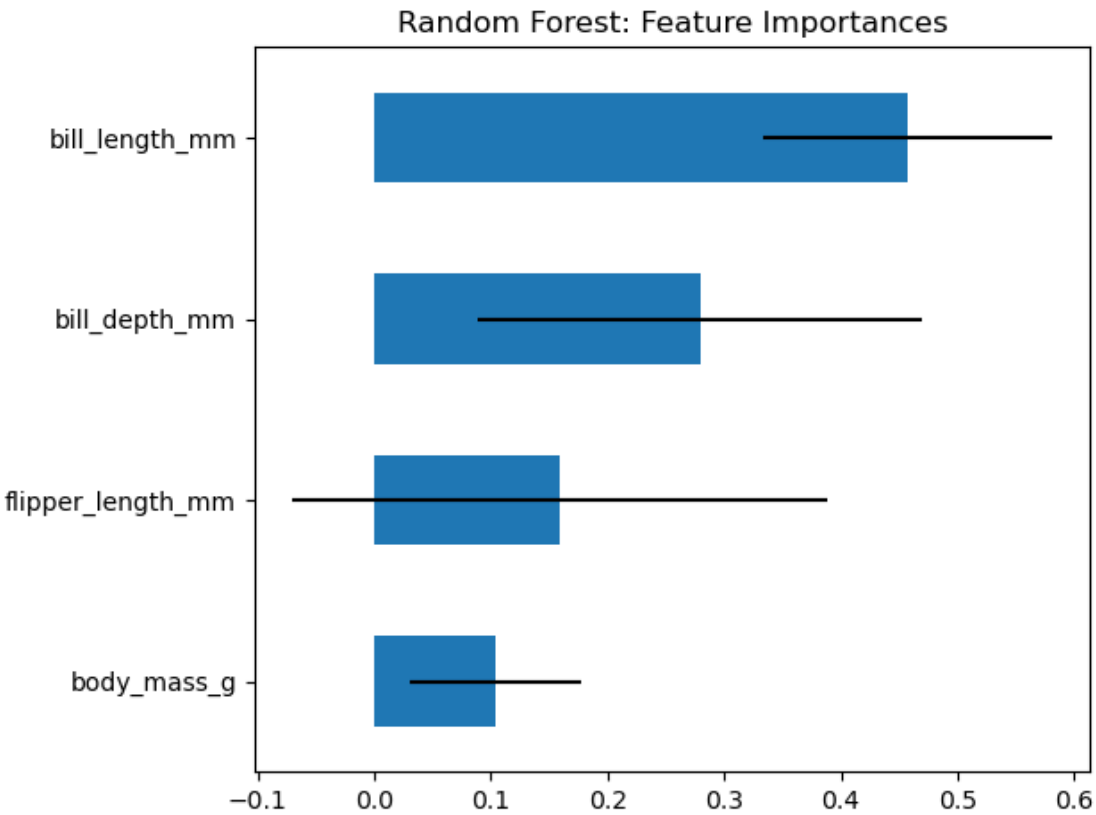


4.2.5 Exercise 12 (CORE)

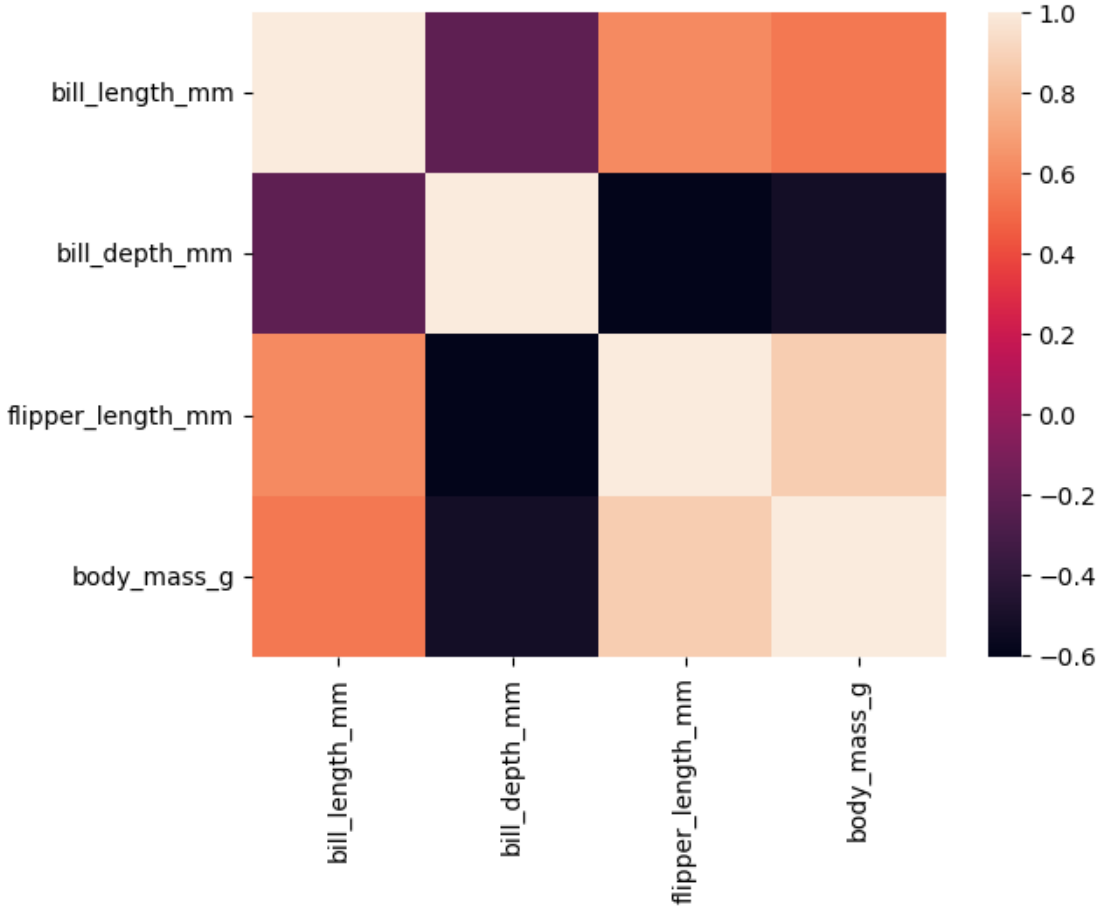
- Run the following code to plot the feature importance (averaged across all trees along with the standard deviation).
- Visualize the correlation matrix of the features.
- Has the ranking of the features changed comparing the pruned tree to the random forest? Does the correlation matrix provide any insight on why?

[58]:
```
# Note: change rf to the name your random forest object in exercise 12

# Compute the standard deviation
std = np.std([tree.feature_importances_ for tree in rf.estimators_], axis=0)

# Create a pandas series with the feature importances
importances_rf = pd.Series(
    rf.feature_importances_, index=feature_names
```

```
).sort_values(ascending=True)

# Plot the feature importances
ax = importances_rf.plot.barh(xerr=std)
ax.set_title("Random Forest: Feature Importances")
ax.figure.tight_layout()
```



[59]:
```
# Code for your answer here
cor=X_train.corr()
sns.heatmap(cor)
plt.show()
```



random forest tree have mpre branch, thus high related feature may chose by different branch and causing more mistakes.

4.3 Voting Classifier

While bagging and random forests focused on creating an ensemble of the same estimator (e.g. trees) fit on different random subsets of the data, the idea behind **voting** is to combine different estimators. The predictions then can be aggregated either by using a majority vote or by averaging predicted probabilities (soft vote). This can be useful for a set of equally well performing models in order to balance out their individual weaknesses.

In the `VotingClassifier`, the default is to aggregate predictions with **hard voting** (i.e. `voting='hard'` by default), which simply looks at the tally of votes received by each class across the estimators in the ensemble. Instead, **soft voting** (i.e. `voting='soft'`) averages the predicted probabilities across the estimators in the ensemble.

For further details, please see the documentation from here: https://scikit-learn.org/stable/modules/ensemble.html#voting-classifier

4.3.1 Exercise 13 (EXTRA)

- Create a voting classifier that combines your pruned tree, random forests, logistic regression, and support vector classifier.
- How does the accuracy on the test data compare across each model individually and the voting classifier?

[ ]:
```
from sklearn.ensemble import VotingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression

# Code for your answer here
```

Type your answer here

5 Competing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change 'Student 1' and 'Student 2' at the top of the notebook to include your name(s)**.

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF. Once generated, please submit this PDF on Learn page by 16:00 PM on the Friday of the week the workshop was given.

[ ]:
```
!jupyter nbconvert --to pdf mlp_week08.ipynb
```